

Code Review Report

Language: python | Review mode: hybrid | Model: llama-3.3-70b-versatile | Generated: 2026-08-13 17:32 UTC

Overall Assessment

The code review has identified 12 findings with a risk score of 45, indicating a high level of risk. The majority of the findings are critical, highlighting the need for immediate attention to prevent potential security breaches.

Merge readiness: Not Ready

Code health: 0/100

Findings Summary

Total findings: 12

Risk score: 45

Severity breakdown: Low 0, Medium 0, High 3, Critical 9

Type	Severity	Line	Message	Recommendation
Hardcoded Password	High	1	password = "admin123"	Store passwords in environment variables.
Hardcoded API Key	High	3	api_key = "123456789"	Store secrets securely.
SQL Injection	Critical	7	query = "SELECT * FROM users WHERE name='" + name + "'"	Use parameterized queries.
Dangerous Dynamic Execution	Critical	9	eval(input())	Avoid dynamic code execution.
Dangerous Dynamic Execution	Critical	11	exec("print('Hello')")	Avoid dynamic code execution.
Command Injection	Critical	15	subprocess.run("dir", shell=True)	Do not use shell=True with user input.
Security	Critical	1	Hardcoded sensitive information (password)	Store sensitive information securely using environment variables or a secrets manager.
Security	Critical	3	Hardcoded sensitive information (api_key)	Store sensitive information securely using environment variables or a secrets manager.
Security	Critical	7	SQL injection vulnerability due to string concatenation	Use parameterized queries or prepared statements to prevent SQL injection.
Security	Critical	9	Arbitrary code execution vulnerability due to eval() function	Avoid using eval() and instead use safer alternatives such as ast.literal_eval() or json.loads().

Type	Severity	Line	Message	Recommendation
Security	Critical	11	Arbitrary code execution vulnerability due to exec() function	Avoid using exec() and instead use safer alternatives such as subprocess.run() with a whitelist of allowed commands.
Security	High	13	Potential command injection vulnerability due to shell=True	Use subprocess.run() with a list of arguments instead of a string, and avoid using shell=True whenever possible.

Remediation Roadmap

Finding	Severity	Impact	Suggested Action
SQL Injection	Critical	High	Replace string concatenation with parameterized queries
Dangerous Dynamic Execution	Critical	High	Replace eval(input()) with a safer alternative, such as a conditional statement or a dictionary lookup
Command Injection	Critical	High	Replace shell=True with shell=False and pass the command as a list of arguments
Security	Critical	High	Replace hardcoded sensitive information with environment variables or a secrets manager
Hardcoded Password	High	Medium	Store passwords in environment variables
Hardcoded API Key	High	Medium	Store secrets securely
Security	High	Medium	Use subprocess.run() with a list of arguments instead of a string, and avoid using shell=True whenever possible

Source Code Snapshot

```
password = "admin123"

api_key = "123456789"

name = input()

query = "SELECT * FROM users WHERE name='" + name + "'"

eval(input())

exec("print('Hello')")

import subprocess

subprocess.run("dir", shell=True)
```